

Python #54

На попередньому занятті ми додали реєстрацію користувачів. Тепер настав час зробити їхню авторизацію використовуючи базу даних. Загальний зміст авторизації полягає в тому, що на сайті є сторінки, доступні як всім користувачам, так і ті, які доступні тільки авторизованим користувачам(Тема 48).

Для цього ми частково модифікуємо попередньо написаний код для користувачів а саме додамо конект до бази даних і отримання даних з неї

Зміни в коді для класу `UserLogin`

До	Після
<pre>class UserLogin: def fromDB(self, user_id) : for people in peopledb: if (user_id == people.id): self.__user = people return self return self def create(self, user): self.__user = user return self def is_authenticated(self): return True def is_active(self): return True def is_anonymous(self): return False def get_id(self) : return self.__user.nickname</pre>	<pre>class UserLogin: def fromDB(self, user_id, db) : self.__user = db.getUser(user_id) return self def create(self, user): self.__user = user return self def is_authenticated(self): return True def is_active(self): return True def is_anonymous(self): return False def get_id(self) : return str(self.__user['id'])</pre>

Тут перший метод `fromDB` використовується для створення об'єкта в декораторі `user_loader`. Він `user_id` виконує завантаження даних з БД і зберігає в приватній властивості `__user`. Другий метод `create` використовується при створенні об'єкта на момент авторизації користувача. Вся інформація про нього вже відома і ми її просто передаємо за посиланням `user` і зберігаємо також у приватній змінній

`__user`. Ця інформація потім стане в нагоді в методі `get_id`, який повертає `id` поточного користувача.

Далі нам потрібно відразу ж визначити метод `getUser` в класі `FDataBase`. Він буде наступним:

```
def getUser(self, user_id):
    try:
        self.__cur.execute(f"SELECT * FROM users WHERE id = {user_id} LIMIT 1")
        res = self.__cur.fetchone()
        if not res:
            print("User not found")
            return False

        return res
    except sqlite3.Error as e:
        print("Error get data form database " + str(e))

    return False
```

```
SELECT * FROM users WHERE id = {user_id} LIMIT 1
```

Дана команда витягає з таблиці `users` всі поля для зазначеного ID користувача і, потім, повертає прочитані дані. Якщо дані прочитати не вдалося, метод `getUser` повертає значення `False`.

Також нам необхідна функція яка буде отримувати дані про користувача, при його вході, за унікальним параметром в основному це є його пошта.

```
def getUserByEmail(self, email):
    try:
        self.__cur.execute(f"SELECT * FROM users WHERE email = '{email}' LIMIT 1")
        res = self.__cur.fetchone()
        if not res:
            print("User not found")
            return False

        return res
    except sqlite3.Error as e:
        print("Error get data form database " + str(e))

    return False
```

Як ми бачимо команда запиту практично не відрізняється від попередньої, різниця полягає лише в тому по якому полі відбувається пошук.

Тепер наш обробник авторизації буде мати наступний вигляд:

```
@app.route('/login', methods=['POST', 'GET'])
def login():
    db = get_db()
    dbase = FDataBase(db)
    if request.method == 'POST':
        user = dbase.getUserByEmail(request.form['typeEmailX'])
        if user and check_password_hash(user['password'], request.form['typePasswordX']):
            userlogin = UserLogin().create(user)
            login_user(userlogin)
            return redirect(url_for('index'))

        print("Invalid login/password pair")

    return render_template('login.html')
```

Також часткових змін зазнає функція `load_user` у ній необхідно додати підключення до бази даних та передати це у клас `UserLogin`

```
@login_manager.user_loader
def load_user(user_id):
    print("load_user")
    db = get_db()
    dbase = FDataBase(db)
    return UserLogin().fromDB([user_id, dbase])
```

LOGIN

Please enter your login and password!

Skool@gmail.com

Email

.....

Password

[Forgot password?](#)

Login

Після входу вас повинно перекинути на головну сторінку так як даний редірект налаштований в функції авторизації (`return redirect(url_for('index'))`), а також в консолі наступний напис:

```
127.0.0.1 - - [18/Nov/2022 13:44:50] "GET /socket.io/?EI0=4&transport=p
load_user
127.0.0.1 - - [18/Nov/2022 13:44:50] "GET /login HTTP/1.1" 200 -
```

Завдання 1.

Додати даний функціонал до розробленого вами сайту на передніх заняттях. Також залишити перевірки полів на коректність даних перед запитом до бази даних. Якщо немає то створити, тобто перевірити чи в полі де має бути Email дійсно він а не набір випадкових символів.

Завдання 2.

Зберегти зміни на git.